Flight Booking Decision Assistant

CS664 Term Project | Prepared for Sheung Wan Wong | October 14, 2026

1. Project goal and simulator

This Python tabular Q-learning application chooses BOOK or WAIT for a simulated traveler with a fixed departure date. It tests whether current fare and time remaining can reduce expected purchase cost relative to simple booking rules. The approach trains a table of BOOK/WAIT values and compares the learned policy (a rule choosing actions from the current state) with three simple booking rules. It also checks the policy against the optimum under the simulator's Markov decision process (MDP), a model of states, actions, and transition probabilities. Supplementary experiments examine training budget and learning rate separately from the original comparison.

State is (time remaining, current price). Seven daily purchase opportunities run from time 6 to 0; fares are $120, $180, $240, $300, and $360. Each episode starts at one of these five prices, with an equal probability of 20% for each. When the agent waits, the next price is sampled using the probabilities for the current time remaining.

Time before WAIT	+$60	Unchanged	−$60
6–3	30%	30%	40%
2–1	60%	20%	20%

WAIT gives reward 0 and reduces time by one. Moves beyond $120–$360 stay at the boundary. BOOK gives the negative current fare as reward and terminates. At time 0, BOOK is mandatory and a requested WAIT becomes BOOK. Actions after purchase raise an error until reset. Refunds, cancellations, and sold-out events are excluded. All probabilities are hypothetical.

2. Training and Q-value updates

Q-learning follows Watkins and Dayan (1992). Q(state, action) estimates the total reward from taking that action in that state, then following the best continuation represented by the current Q-table. Each training episode is one simulated trip ending in a purchase. Rewards are negative purchase costs, so a larger Q-value is better. The table starts at zero; the agent learns from observed transitions without receiving their probabilities.

Original setting	Value and meaning
Training episodes / seeds	50,000 trips; training seed 66401, test seed 66402. Seeds initialize reproducible random-number streams.
Alpha (learning rate)	0.1; controls how much each new target changes the current estimate.
Gamma (discount factor)	1.0; future rewards are not discounted.
Epsilon and its schedule	Epsilon is the probability of a random action during training. It decreases linearly from 1.0 initially to 0.05 by episode 40,000, then stays at 0.05.
Greedy evaluation	Choose the largest legal Q-value; ties choose BOOK. Epsilon 0 means no random exploration. No Q updates means the trained table stays fixed throughout evaluation.

2. Training and Q-value updates (continued)

Each update changes the CURRENT state/action entry: Q_new = Q_old + alpha × (target − Q_old). The target is the observed reward for terminal BOOK: the trip has ended, so there is no future value. For nonterminal WAIT, target = reward + gamma × the largest legal Q-value at the observed next state. At a deadline next state, only BOOK is legal. The next state is observed after acting; no future price is known in advance.

Actual BOOK update — original training log

Episode 1, current state (6, $120), action BOOK. Observed reward = -120; old current Q = 0; target = -120; alpha = 0.1; updated current Q = -12. This purchase ends the episode, so no next-state value is used. Arithmetic: 0 + 0.1 × (−120 − 0) = −12.

The BOOK record is saved in results/run_summary.json. No complete actual WAIT update with its observed next state and historical next-state Q-values was found in the saved logs, Notebook outputs, or result artifacts. The example below uses assumptions, not reconstructed training data.

Illustrative WAIT update — assumed Q-values, not an actual training log

Component	Hypothetical value
Current state / action	(1, $240) / WAIT
Observed next state in this example	(0, $180)
Immediate reward	0
Assumed next-state legal Q-values	Q(BOOK) = −180; WAIT is illegal at the deadline.
Assumed old current Q(WAIT)	−200
Gamma / alpha	1 / 0.1
Target	0 + 1 × (−180) = −180
Updated current Q(WAIT)	−200 + 0.1 × [−180 − (−200)] = −198

This update changes Q((1, $240), WAIT), not the next-state entry. Ticket prices are limited to $120–$360, while Q-values are learned reward estimates and need not equal a valid ticket price. The old value −200 and next-state value −180 here are assumed estimates.

The complete table on the next page contains final values after 50,000 training episodes, from results/q_values.npy. There are 7 time values × 5 prices × 2 actions = 70 stored entries, with 65 legal state/action pairs. Deadline WAIT is not allowed. Values are rounded to three decimals; greedy actions use unrounded values. This final learned table is separate from the logged BOOK update and illustrative WAIT example above. Even BOOK estimates can differ from the known negative fare after finite training.

For example, at (6, $120), Q(BOOK) = -120.000 exceeds Q(WAIT) = -148.415, so the policy BOOKs. At (6, $300), Q(WAIT) = -268.136 exceeds Q(BOOK) = -300.000, so it WAITs. The larger (less negative) legal estimate determines the action; these estimates do not establish optimality.

2. Complete original trained Q-table

Time remaining	Price	Q(BOOK)	Q(WAIT)	Greedy action
6	$120	-120.000	-148.415	BOOK
6	$180	-180.000	-181.837	BOOK
6	$240	-240.000	-225.957	WAIT
6	$300	-300.000	-268.136	WAIT
6	$360	-360.000	-293.377	WAIT
5	$120	-120.000	-132.190	BOOK
5	$180	-180.000	-182.415	BOOK
5	$240	-240.000	-231.913	WAIT
5	$300	-300.000	-273.533	WAIT
5	$360	-360.000	-304.489	WAIT
4	$120	-120.000	-138.124	BOOK
4	$180	-180.000	-181.505	BOOK
4	$240	-240.000	-247.487	BOOK
4	$300	-300.000	-291.629	WAIT
4	$360	-360.000	-312.530	WAIT
3	$120	-120.000	-140.532	BOOK
3	$180	-180.000	-190.639	BOOK
3	$240	-240.000	-248.993	BOOK
3	$300	-300.000	-303.299	BOOK
3	$360	-360.000	-326.366	WAIT
2	$120	-120.000	-153.879	BOOK
2	$180	-180.000	-208.834	BOOK
2	$240	-240.000	-261.968	BOOK
2	$300	-300.000	-327.361	BOOK
2	$360	-360.000	-335.198	WAIT
1	$120	-119.734	-155.873	BOOK
1	$180	-179.998	-208.205	BOOK
1	$240	-240.000	-256.198	BOOK
1	$300	-300.000	-313.791	BOOK
1	$360	-360.000	-348.515	WAIT
0	$120	-108.183	Not allowed	BOOK
0	$180	-179.788	Not allowed	BOOK
0	$240	-239.958	Not allowed	BOOK
0	$300	-300.000	Not allowed	BOOK
0	$360	-360.000	Not allowed	BOOK

3. Original sampled four-strategy comparison

Only Q-learning was trained: 50,000 episodes with seed 66401. The three baselines are fixed rules requiring no training. All four used the same 10,000 fresh seven-price test paths, generated with NumPy seed 66402. Only the evaluator sees full paths; policies see the current state and book by the deadline. The $180 threshold was the cheapest of the five fare-level fixed thresholds under the known model, without using test results. The learning agent does not receive that model.

Strategy	Rule	Sample mean cost
Immediate booking	BOOK first day	$239.58
Minimum-price rule ($120)	BOOK at $120; else deadline	$223.48
Fixed-threshold rule ($180)	BOOK at $180 or below; else deadline	$219.74
Q-learning (greedy)	Original greedy policy	$214.85

Q-learning had the lowest sampled mean ($214.85), with mean paired savings of $4.88 versus the strongest rule, the $180 threshold.

Baseline	Mean paired saving	Individual paired 95% CI
Immediate booking	$24.73	[$23.84, $25.61]
Minimum-price rule ($120)	$8.62	[$7.24, $10.00]
Fixed-threshold rule ($180)	$4.88	[$4.03, $5.74]

For 10,000 test paths, paired saving is baseline cost minus Q-learning cost on the same path. Each 95% confidence interval (CI) places a margin around the mean: the paired sample standard deviation divided by the square root of 10,000, multiplied by SciPy's Student-t multiplier (97.5th percentile; 9,999 degrees of freedom). All endpoints are positive, supporting mean savings conditional on this trained policy. These large-sample intervals do not describe individual-trip savings, training variation, or simultaneous coverage across baselines.

Evidence: results/mean_cost_comparison.csv, results/paired_savings_95ci.csv, and results/paired_purchase_costs.csv.

4. Exact model expectations

Backward induction starts at the deadline fare and works backward, choosing booking or the probability-weighted cost of waiting with optimal continuation. Fixed-policy evaluation follows a policy's actions instead. Both average over the five equally likely starting fares. These are analytical expectations to floating-point precision, not sampled means or Q-estimates. The optimality gap is learned-policy expected cost minus optimal expected cost.

Analytical reference	Expected cost
Optimal finite-horizon policy	$207.274224
Original learned policy	$215.709120
Learned minus optimal	$8.434896

The learned policy remains $8.43 above optimal per trip: at (6, $180), it BOOKs while the optimum WAITs. Its $214.85 sample mean estimates the $215.709120 model cost with sampling variation. Evidence: results/exact_mdp_comparison.csv and results/exact_state_values.csv.

5. Original learned policy and complete decision trace

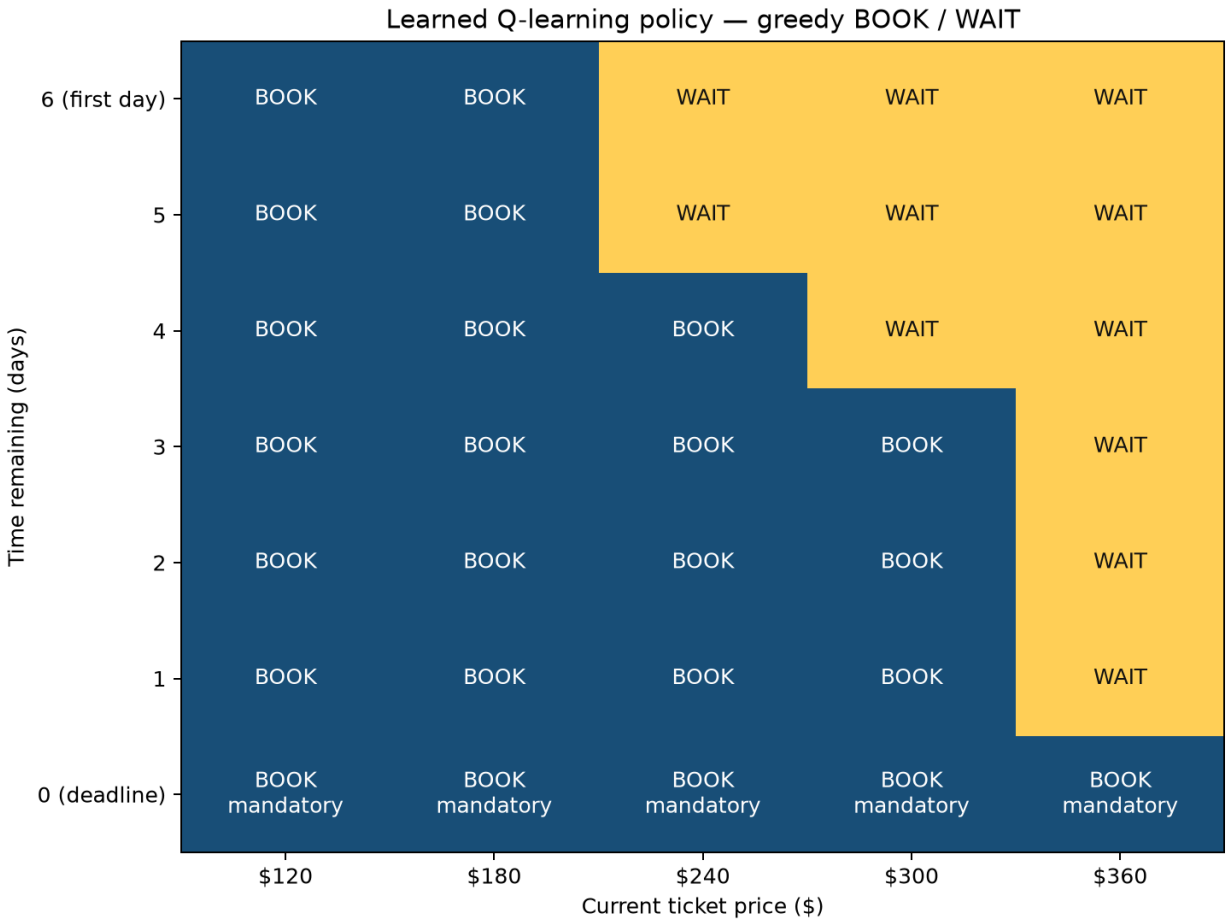


Figure 1. Original greedy policy: BOOK or WAIT at each state. Time 0 is mandatory BOOK.

The heatmap shows time-dependent decisions: at $240, WAIT at times 6–5 changes to BOOK at 4–0; $120 is always BOOK. This describes the learned rule, not proof of optimality. The complete trace is path index 1, the first test example with at least three decisions; unseen post-purchase prices are excluded.

Day	Time remaining	Price	Action	Reward
1	6	$360	WAIT	0
2	5	$300	WAIT	0
3	4	$360	WAIT	0
4	3	$360	WAIT	0
5	2	$360	WAIT	0
6	1	$360	WAIT	0
7	0	$360	BOOK	-360

The episode ends after 7 decisions with a $360 purchase. It illustrates policy behavior, not average performance: positive mean savings can coexist with expensive trips. Evidence: results/q_learning_trace.csv and results/learned_policy_actions.csv.

6. Executed verification

Core check	Cases	Outcome
simulator transitions	90	PASS
episode rules	7	PASS
Q-value updates	2	PASS
exact MDP reference	18,225	PASS
policy actions	5	PASS
paired intervals	2	PASS
future-price isolation	400	PASS

The exact reference was independently checked by enumerating all 3,645 possible starting-price/move sequences for each of five policies (18,225 comparisons), weighted by their model probabilities. Supplementary checks passed for nonzero-old-value Q updates, deadline selection, all-state rewards and invalid actions, correct identification of optimal-action ties, independent training, and preservation of the original model. Actions with equally low exact expected costs are treated as optimal ties.

Additional logging checks verified update counts, applied learning rates, terminal/legal targets, and evaluation at checkpoints. At all 300 checkpoints (saved training snapshots), evaluation left Q, update counts, and the random-number generator state unchanged. Constant alpha 0.1 exactly reproduced the ten earlier 50,000/200,000 endpoint Q-tables, visit counts, and analytical costs. The saved main Notebook has 80 successfully executed code cells. These finite checks support implementation behavior; they do not prove convergence.

Complete executed records: results/verification_results.csv, results/supplementary_verification.csv, and results/learning_dynamics_verification.csv.

7. Supplementary training-budget experiment

Seeds 66401, 66411, 66421, 66431, and 66441 each restarted from zero at 50,000 and 200,000 episodes (ten runs). Only episode budget changed: simulator, alpha 0.1, gamma 1, initialization, and the original epsilon schedule stayed fixed. Final greedy policies were evaluated exactly under the uniform initial-fare distribution.

Episodes	Mean exact cost	Across-seed cost range
50,000	$216.87	[$215.45, $219.97]
200,000	$220.98	[$219.52, $221.91]

All five seeds worsened: mean exact cost rose from $216.87 to $220.98 (+$4.11). More episodes alone did not improve these constant-0.1 runs; seed 66401 increased by $4.26. All ten models cost less than the immediate and $120 baselines in exact evaluation; seven cost less than the $180 baseline. All 65 legal pairs were visited, yet seven original-policy states were suboptimal. None had an action in the bottom quarter of visit counts; this does not explain the gap.

Full evidence: results/supplementary_training_runs.csv, results/supplementary_paired_gap_changes.csv, results/supplementary_suboptimal_states.csv, and CS664_Flight_Booking.ipynb.

8. Supplementary learning-rate experiment

The five seeds restarted independently for 200,000 episodes under constant alpha 0.1, constant alpha 0.03, or alpha(s,a) = 0.1/(1 + N_previous(s,a)/1000). N_previous counts prior updates to that pair. With 0, 1,000, and 3,000 prior updates, alpha is 0.1, 0.05, and 0.025. Scale 1,000 was prespecified. Other settings and the legal greedy evaluator were unchanged.

Fifteen runs recorded 300 training checkpoints and 2,100 state records for the seven originally suboptimal states. Equal seeds do not ensure identical experience across conditions: changed policies can alter later experiences and consumption of random numbers. Every run, including equal-cost policies, remains in the evidence.

Condition	Mean exact cost	Across-seed cost range	Mean optimality gap
Constant 0.1	$220.98	[$219.52, $221.91]	$13.70
Constant 0.03	$213.32	[$209.96, $215.93]	$6.05
Visit decay	$209.16	[$207.93, $209.67]	$1.89

Visit decay had the lowest tested mean exact cost ($209.16, versus $220.98 for constant 0.1), but its mean optimality gap remained $1.89. For each of the five seeds, both alternative learning rates lowered final exact cost relative to constant alpha 0.1. For seed 66401, reductions were $10.01 with alpha 0.03 and $10.29 with visit decay. Every final policy still had a positive optimality gap. Checkpoint costs fluctuated in all conditions: for example, seed 66401 under alpha 0.1 worsened by $4.72 from 20,000 to 30,000 episodes. Action switches accompanied performance changes, but sparse checkpoints do not isolate their cause. No best-checkpoint selection was performed.

Full run and checkpoint evidence: results/learning_dynamics_final_comparison.csv, results/learning_dynamics_paired_comparisons.csv, and results/learning_dynamics_checkpoints.csv. The saved checkpoint plot, results/learning_dynamics_checkpoint_gaps.png, shows all five seeds per condition; lines join recorded evaluations only. RESULTS.md indexes state diagnostics.

9. Limitations and validation answers

Cost reduction: yes in the original simulator test sample, relative to all three baselines; an individual trip need not save money. Reliability: the paired intervals support positive mean savings for this fixed policy; the exploratory seed studies do not establish universal robustness. Optimality and correctness: the original policy remains above the exact optimum, while saved verification supports the stated implementation without proving convergence.

The hypothetical probabilities, five fares, short horizon, and omission of sold-out events provide no evidence of real-flight savings. Across-seed ranges are descriptive, not confidence intervals; two budgets do not establish a continuous trend. The supplementary analyses are exploratory, not independent confirmation. Learning rates can affect target variability, learning speed, and state/action coverage, but their causal contributions were not isolated. No additional tuning was performed.

Project evidence files are included in the submitted ZIP; paths are relative to its top-level folder. README.md provides setup and execution instructions; RESULTS.md indexes complete saved outputs. Monetary summaries are rounded after full-precision calculations; saved artifacts retain their precision.